

6.4110 Miniproject 4Collaborators: *Claude helped me format the L^AT_EX file***1. Fire Problem**

(i) The components of a state are:

- $M \times N$ possible robot locations
- 2 possible values for `carried_patient`
- $2^{M \times N}$ possible values for the state of the fire grid

This gives a total state space of size $M \times N \cdot 2^{MN+1}$, which is enormous. If we approached this with dynamic programming, at every relevant t we would need to compute the optimal policy for each of these states, which is infeasible.

(ii) We compute $P(F_{ij}^{(t+1)} \mid F^{(t)})$. Let F_{neighs} be the random sub-matrix of $F^{(t)}$ containing the neighbors of cell (i, j) and the cell itself. Let F_r range over $\{0, 1\}^{|F_{\text{neighs}}|}$, the set of all such realized matrices. Let $K_{i'j'} = W[i' - i + 1, j' - j + 1]$.

Using the formulas given in the problem:

$$\begin{aligned}
 P(F_{ij}^{(t+1)} \mid F^{(t)}) &= P(F_{ij}^{(t+1)} \mid F_{\text{neighs}}) \\
 &= \sum_{F_r} P(F_{\text{neighs}} = F_r) \sum_{i', j'} K_{i'j'}(F_r)_{i'j'} \\
 &= \sum_{i', j'} K_{i'j'} \sum_{F_r} P(F_{\text{neighs}} = F_r) (F_r)_{i'j'}.
 \end{aligned}$$

Since $(F_r)_{i'j'} \in \{0, 1\}$, the inner sum equals $P(F_{\text{neighbor}(i', j') \text{ of } (i, j)}^{(t)} = 1) = F_{i'j'}^{(t)}$. Therefore:

$$P(F_{ij}^{(t+1)} \mid F_{\text{neighs}}) = \sum_{i', j'} K_{i'j'} F_{i'j'}^{(t)} = (W * F^{(t)})_{ij},$$

where $*$ denotes element-wise convolution. Hence

$$P(F^{(t+1)} \mid F^{(t)}) = W \circledast P(F^{(t)}).$$

- (iii) Given an initial state s_0 , let $\pi = (s_1, s_2, \dots, s_T)$ denote a path. We want:

$$\begin{aligned}
 \pi^* &= \arg \max_{\pi} \prod_{i=1}^T P(\text{robot survives step } s_{i-1} \rightarrow s_i) \\
 &= \arg \max_{\pi} \sum_{i=1}^T \log P(\text{step } i \text{ is safe}) \\
 &= \arg \max_{\pi} \sum_{i=1}^T \log(1 - P(\text{onfire}_{s_i})) \\
 &= \arg \min_{\pi} \sum_{i=1}^T -\log(1 - P(\text{onfire}_{s_i})) \\
 &= \arg \min_{\pi} \sum_{i=1}^T c - \log(1 - P(\text{onfire}_{s_i})).
 \end{aligned}$$

We add a constant c so that the cost is never zero: when $P(\text{onfire}_{s_i}) = 0$, $\log(1) = 0$. Step costs of 0 make our search algorithms break.

- (iv) When the agent sees that its path to the hospital is obstructed by fire, it starts pacing between two squares. Only after the path is unblocked does it continue moving towards the hospital.

At each time step the agent searches the space of possible futures to find one where it reaches the goal. It does this by simulating the fire probabilistically – evolving the fire probability grid after every time step. When the probabilities of fire along the chosen path are high, the path cost explodes since $-\log(1-p) \rightarrow \infty$ as $p \rightarrow 1$. The agent then prefers paths through safe squares and waits, circling until it exhausts its step budget.

2. Stochastic Fire Problem

- (i) The agent chooses to go right and manoeuvre around the fire. Introducing stochasticity removes the certainty about where fire exists—especially further into the planning horizon. We also lose certainty about cells that definitely contain no fire. This makes the agent more cautious and more likely to choose longer but safer paths over shorter direct ones.
- (ii) A closed-loop agent would not make a long-term plan based solely on hypothetical rollouts of the system. Although our A* agent is not entirely open-loop – before taking a step, it makes an observation and replans – it still produces a long-term plan by simulating the system (in the sense of propagating probabilities through time) without using the actual future state. The agent's belief can lead it to consider some paths as completely unviable because they have a high *probability* of being dangerous, whereas the actual realisation of the fire is such that those paths are viable and even shorter.

- (iii) If planning were very expensive, I would prefer not to replan after every step. Similarly to the previous mini-project, we could follow an initial plan and observe the environment as we go. At each observation we compute the probability that the next planned step leads the agent into fire. If this probability exceeds a threshold (e.g. 5%), we replan. The trade-off is that by following the original plan blindly we could end up surrounded by fire; when we finally detect this and replan, there may be no remaining viable path.
- (iv) Most-likely-outcome determinization would be problematic. Consider the following initial state (H = hospital, # = fire, a = agent carrying patient, - = wall):

```

. H #
# . #
-
# . a
# . .

```

The agent must choose between going left and going down (a wall blocks the upward route). The cell to the left has four fire neighbors, so the most likely outcome is that it remains clear in the next step—but with only about 44% probability, which is hardly guaranteed. The shorter path goes left, so the determinized agent chooses it. However, moving down first and waiting to see how the fire develops may be the safer strategy before committing to a route.

3. MCTS

- (i) Both agents choose the wide, spacious route in order to avoid potential fire on their path. MCTS is more of a closed-loop agent than A*: it does not plan far into the future but instead tries to predict as many possible futures as possible and acts according to the best estimated reward for each action. In some sense MCTS distinguishes different possible futures better than A* does. A*, by propagating fire probabilities, “mushes together” many potential futures and does not maintain a clear picture of what any single one looks like.
- (ii) The advantage of A* lies precisely in this mushing together. MCTS only knows about the futures it encounters during rollouts: with fewer rollouts it knows less about the ways the MDP can evolve and makes less informed decisions. A*, by contrast, encodes information about *all* possible futures through the per-cell fire probabilities.

In particular, MCTS might roll out roughly equal numbers of futures starting with action *right* and action *down*. However, many more rollouts of action *right* are needed to get a reliable estimate of its value. Given a large rollout budget, MCTS will start realising that *right* yields higher reward and will prioritise it. With small budgets the outcomes are too noisy to reveal this.

- (iii) Depending on how dangerous the surrounding area actually is, the state value might not be affected very much. With $n_{\text{simulations}} = 10$, a single bad rollout from a given state incurs a large negative reward, but this can be compensated by the positive rewards of the remaining nine rollouts. If the area is genuinely dangerous, it is more likely that all rollouts lead to bad outcomes, and the state value reflects that correctly.